We know ML involves learning patterns from data

But how?

Scenario: Predict house prices based on area

Area	**Price**
500 sq.ft	₹25L
1000 sq.ft	₹50L
1500 sq.ft	₹75L
2000 sq.ft	₹100L

Table 1.1

Relationship seems to be: Larger area → Higher price.

But initially, the model doesn't know the correct relationship. It learns.

To understand how it learns, we need to know certain terms.

1. Parameters

A parameter is a variable inside a model such as a weight (w) or bias (b) that the system learns on its own from data.

In simple linear regression: $\hat{y} = wx + b$

x = input

$\hat{y}$ = predicted output

w = weight

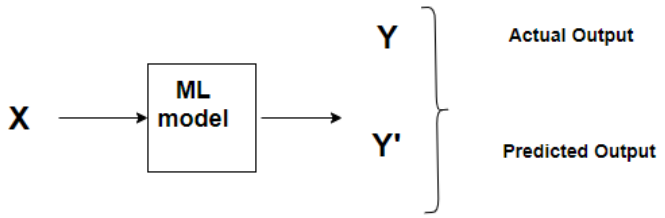
b = bias

initially, w and b has some initial value and the model makes prediction.

---

2. Loss – How wrong the model's predictions are

From table 1.1, For the input 700sq.ft ,it might predict as ₹40L but actual rate is ₹50L. It produces error.



Error = Y'(Predicted) - Y(Actual)

a)Loss function:

gives us a numerical measure of how bad the prediction is. It tells the model that its current parameters are producing this much error.

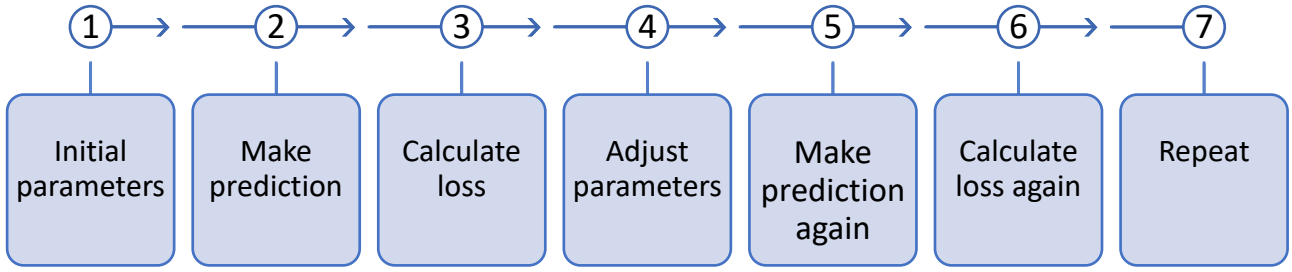
$$Cost = \frac{1}{N}\sum_{i=1}^{N}(Y' - Y)^2$$

The goal of any Machine Learning Algorithm is to minimize the Cost Function.

Lower error = good learning

---

3. Training

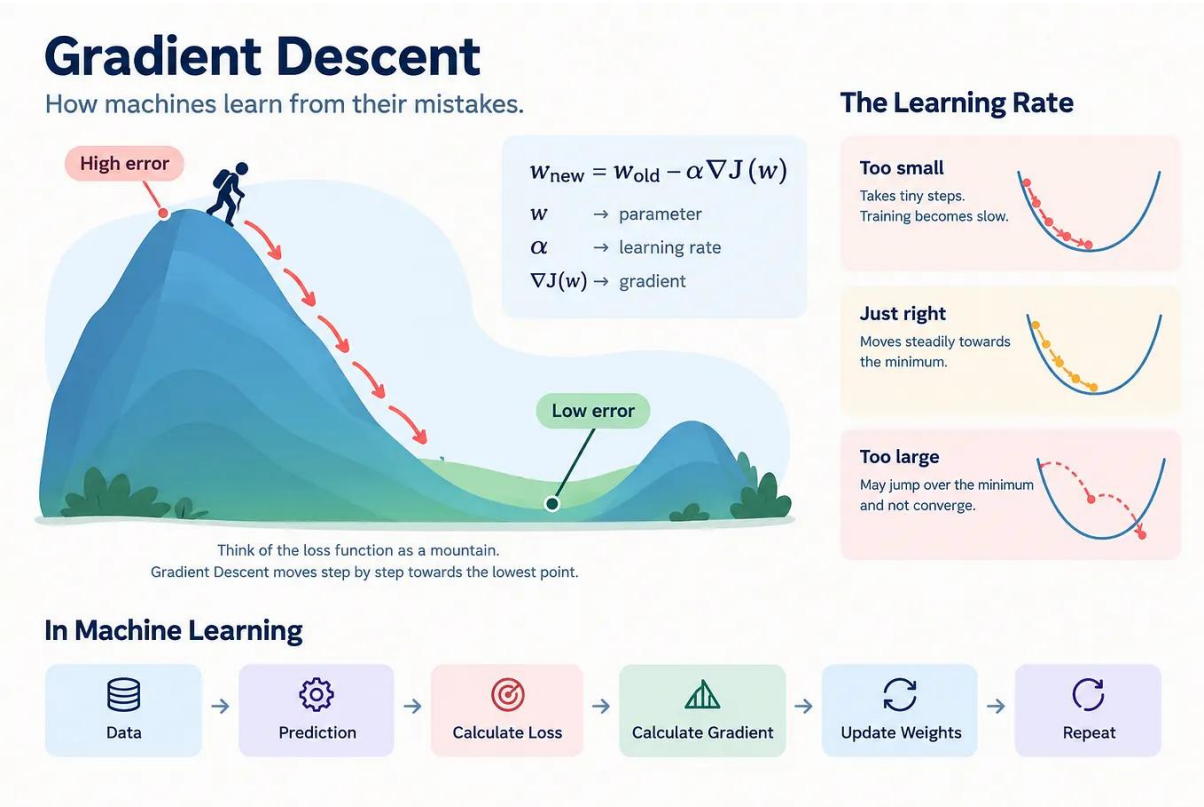


Eventually, we want the model to find parameters that produce relatively low loss. This repeated process is **training**. It is basically an optimization process.

---

4. Gradient Descent

Gradient descent is an optimization algorithm used to adjust model parameters in a direction that reduces the loss**.**



- The Mountain represents the model's Loss Function
- The Valley is the point of minimum error (the perfect settings for the model).
- The Slope you feel is the Gradient (the direction of steepest error increase, so you move in the *opposite* direction to go down).
- Your Step Size is the Learning Rate

---

5)Learning Rate

The learning rate controls roughly how large the parameter updates are.

Too high → might overshoot the valley entirely and bounce back up, failing to learn.

Too Low → The training will take forever and consume too much computer processing time.

The 4-Step Learning Process:

1. **Make a Guess:** The model starts with random settings (parameters) and makes a prediction on the data.
2. **Calculate the Error:** A **Loss Function** compares the guess to the actual correct answer to see how far off it was.
3. **Find the Slope:** The algorithm uses calculus (derivatives) to calculate the **gradient**. This tells the model exactly which direction to change its settings to make the error smaller.
4. **Update Parameters:** The model adjusts its internal settings by subtracting a small fraction of the gradient.

The model loops through these steps thousands of times until the error stops shrinking—a state called **convergence**.

$$\theta_{new} = \theta_{old} - (\alpha \times \text{Gradient})$$

$$\text{New Parameter} = \text{Old Parameter} - (\text{Learning Rate} \times \text{Direction of Slope})$$

---

6)Training terms:

Suppose you have 10,000 training examples

a)Epoch : the model goes through the entire training dataset once, that's one epoch.

Epoch 1 → entire training dataset

Epoch 2 → entire training dataset again

Epoch 3 → again

b) Batch : Instead of processing 10,000 examples at once, we often process smaller groups.

For example: 10,000 examples

Batch 1 → 100 examples

Batch 2 → 100 examples

Batch 3 → 100 examples...

Batch size = number of training examples processed in one iteration/update.

c) Iterations per Epoch = Total Training Examples / Batch Size.

Dataset = 10,000

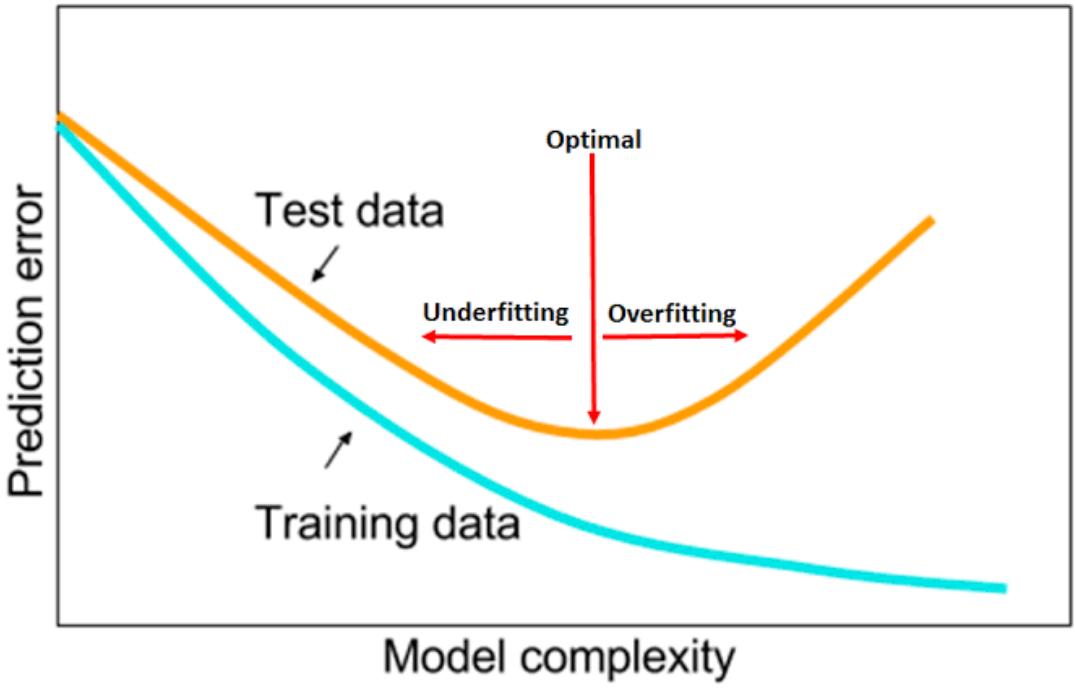
Batch size = 100

Iterations per epoch = 100

---

7) Overfitting vs Underfitting:

In ML, underfitting, good fit and overfitting describe how well a model learns the patterns in your data and how it performs on new, unseen data.



Underfitting

Model hasn't learned enough.

Training performance → poor

Test performance → poor

Good fit

Model learns useful general patterns.

Training → good

Test → good

Overfitting

Model learns the training data too specifically.

Training → extremely good

Test → significantly worse

---

8) Generalization

We separate training and test data so we can measure the model's ability to generalize i.e, perform well on new, unseen data.

GitHub Repository — AI Essentials